Collapse · текущая архитектура

Карта компонентов, хранилищ и границ ответственности по текущим рабочим исходникам.

СБОРКА ПРИЛОЖЕНИЯ — lib/main.dart · lib/app.dart · lib/dependency_injection.dart

- **main → CollapseApp** — Инициализация и восстановление сессии
- **Один GetIt** — Общие репозитории, транспорт и база
- **Один AppRouter / GoRouter** — URL и сохранённые ветки навигации

iPhone · Android · Web
Flutter 3.41.3 / Dart 3.11.1
Desktop — адаптивный web

01 ПРЕДСТАВЛЕНИЕ — lib/src/presentation

- **Вход, контакты, профиль** — HomeScreen · AuthScreen · ProfileScreen
- **Чат и поиск** — ChatScreen · MessageSearchScreen; MessageList → flutter_list_view · Flyer Chat
- **Медиаатлас и просмотрщик** — MediaAtlas · MediaViewer; extended_image · audio / video players

действия пользователя ↓ · ↑ состояние

02 СОСТОЯНИЕ И СЦЕНАРИИ UI — flutter_bloc / Cubit

- **AuthCubit · SettingsCubit · ContactsCubit · ProfileCubit** — Settings → SharedPreferences
- **ChatsCubit · ChatCubit · MessageSearchCubit** — Окно чата ≤ 120 · страницы по 40
- **MediaGalleryCubit** — Собственные курсоры и выбор медиа; Живёт отдельно от окна ChatCubit

команды и запросы ↓ · ↑ изменения

03 РЕПОЗИТОРИИ · ВЛАДЕЛЬЦЫ I/O — lib/src/data

- **AuthRepository** — Сессия, reconnect → SecureSessionStore
- **ProfileRepository** — Профили, контакты, группы; кеш в памяти
- **ChatsRepository** — История и её покрытие · realtime; Outbox: доставка, upload, retry, ACK; Поиск, черновики, read / recv, unread
- **MediaGalleryRepository** — Метаданные → gateway.media / Drift
- **MediaRepository** — HTTP-байты; native disk / web RAM cache

транзакции и чтение ↓ · протокол и upload ↓

04 ЛОКАЛЬНЫЕ ДАННЫЕ — lib/core/database

- **Drift / SQLite · AppDatabase** — Schema 3 · данные разделены по server + account; Topics / Messages: история, черновики, receipts; Outbox / OutboxAttachments: очередь и байты; MessageSearch + FTS5 / MediaEntries: индексы

04 ПРОТОКОЛ И КОДЕКИ — packages/tinode

- **TinodeGateway + DraftyCodec** — WebSocket JSON: auth, topics, history, receipts, media; HTTP upload; канонический Drafty на диске и в сети; Пакет не владеет UI и базой приложения; Скачивание байтов выполняет MediaRepository

СЕРВЕР
collapse-backend

Caddy · только web
Flutter assets и URL rewrite
/v0/ HTTP + WebSocket
Same-origin proxy → Tinode

Tinode v0.25.3
Auth, сообщения, подписки
История и read / recv
Обычные uploads / файлы

+ Media overlay
get.what=media → meta.media

хранение ↓

Хранилища Tinode
PostgreSQL: сообщения, ACL
+ 3 индекса для медиа
Файлы: uploads / вложения

HTTP download из MediaRepository
WS / HTTP upload из Gateway

Native → Tinode напрямую

Общие модели: lib/src/domain/chat · Одна Material-тема + EN/RU · PushService опционален и по умолчанию выключен.

Стрелки показывают основные потоки между слоями; вторичные связи опущены. Источники: ARCHITECTURE.md, dependency_injection.dart и код владельцев.

Как проходят данные

Три отдельных контура: доставка сообщения, чтение истории и просмотр медиа.

A ОТПРАВКА СООБЩЕНИЯ

Сначала локальный commit, затем сеть

Композиция
ChatScreen → ChatCubit
Markdown → Drafty

→

Локальная запись
ChatsRepository → Drift
Сообщение + байты + outbox

граница сохранения

Outbox worker
Внутри ChatsRepository
Читает очередь из Drift

→

TinodeGateway
HTTP upload → refs
WS publish + attachments

→

ACK / echo
Атомарный merge в Drift
client ID остаётся прежним

ACK = подтверждение
Exactly-once не гарантируется

Очередь переживает рестарт и закрытие экрана. После commit UI получает обновление; доставка не ждёт отрисовки.

B ИСТОРИЯ И ОКНО ЧАТА

Кеш → готовый снимок → фоновая сверка

ChatCubit
open / older / newer / jump
Навигация и поколение запроса

→

ChatsRepository
Drift: кеш + известные диапазоны
При пробеле → Gateway.history

→

ChatCubit: HistoryWindow
Страница 40 · окно до 120
Готовый согласованный снимок

→

MessageList
flutter_list_view: ключи и позиция
MessageLayout: геометрия текста

Из окна можно вытеснить строки; из SQLite они не удаляются. Неизвестный офлайн-пробел = blocked.

READ Видимое входящее + активные app / route / pane → ChatCubit → ChatsRepository → Drift + gateway.note

C МЕДИА: МЕТАДАННЫЕ ОТДЕЛЬНО ОТ БАЙТОВ

Независимо от ChatCubit; офлайн-кеш частичный

Атлас / просмотрщик
MediaAtlas / MediaViewer
MediaGalleryCubit: свои курсоры

→

MediaGalleryRepository
Кеш метаданных: MediaEntries
Пакет media через chats.gateway

→

gateway.media → Tinode
get.what=media → meta.media
ACL + индексы PostgreSQL

MediaRepository
Оригиналы: auth HTTP + кеш
Native: диск · Web: RAM

при открытии оригинала

meta.media не меняет историю и read / recv.

Pending-байты → ChatsRepository

СКВОЗНОЕ ВЛАДЕНИЕ СОСТОЯНИЕМ

URL и жизненный цикл
GoRouter + ConversationShell
Resize сохраняет Cubit и черновик

Окно, позиция, эффекты
ChatCubit / MessageList / motion
Три владельца; эффекты — paint-only

Активность экрана
ViewActivity + route + app
Скрытие останавливает playback/read

Поиск по локальному кешу
MessageSearchCubit → ChatsRepository
Drift FTS5; свой курсор результатов

Непрерывные стрелки — основной ход операции; пунктир — условная ветка. При смене сессии устаревшую работу отсекают generation-проверки.

Сверено с message_queue.dart, outbox_delivery.dart, chat_cubit.dart, media_gallery_repository.dart, AppDatabase и тестами. Это схема кода, не отчёт о новом прогоне тестов.